

Week10_예습과제_이재린

≡ 태그

▼ ch5. 토큰화

자연어와 토큰화

1. 자연어

- 자연어란? 사람들이 쓰는 자연히 만들어진 언어(<>인공적으로 만들어진 프로그래밍 언어)
- 자연어 처리(NLP) : 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
 - 목표 : 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하도록
 - 알고리즘이 해결해야 할 문제 : 모호성(맥락을 이해하고 구분), 가변성(사투리, 강세, 신조어 등), 구조(문장의 구조와 문법적 요소)

then how?

2. 토큰화

- **말뭉치**(자연어 모델을 훈련하고 평가하는데 사용되는 대규모의 자연어)를 일정한 단위인 **토큰**(개별 단어나 문장 부호 등 더 작은 단위)으로 나누어야함.

→ 토큰화

- 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
- 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', 'ㄹ', '수', '있', '다', '.']

- 토큰나이저 : 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 SW
 - 공백 분할
 - 정규 표현식 적용
 - 어휘 사전 적용 : 사전에 정의된 단어를 활용 → OOV 존재 가능성 O & 차원의 저주(희소 데이터로 표현될 가능성 O)
 - 머신러닝 활용

단어 및 글자 토큰화

단어 및 글자 토큰화가 비교적 간단하고 명료하지만 꽤 강력함

1. 단어 토큰화

- 단어 토큰화 : 전처리 작업, 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업 → 띄어쓰기, 문장 부호, 대소문자 등 **특정 구분자**를 활용

for : 품사 태깅, 개체명 인식, 기계 번역

```
# 5-1. 단어 토큰화
```

```
review="현실과 구분 불가능한 cg, 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
```

```
tokenized=review.split()
```

```
print(tokenized)
```

```
['현실과', '구분', '불가능한', 'cg,', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

→ 공백을 기준으로 나눈다

→ cg와 붙은 마침표처럼 OOV에 취약 & 한국어 접사, 문장 부호, 오타나 띄어쓰기 오류 등에 취약

2. 글자 토큰화

- 글자 토큰화 : 글자 단위로 문장을 나누는 방식, 작은 단어 사전 구축 가능
 - 작은 단어 사전 : 전체 말뭉치 학습시 각 단어를 더 자주 학습 가능
- for : 언어 모델링(다음 문자를 예측)과 같은 시퀀스 예측 작업

#5-3. 자소 단위 토큰화

```
from jamo import h2j, j2hcj
review="현실과 구분 불가능한 cg, 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed=j2hcj(h2j(review))
tokenized=list(decomposed)
print(tokenized)
```

```
['ㅎ', 'ㅣ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅍ', 'ㅣ', 'ㄱ', 'ㅌ', 'ㅂ', 'ㅌ', 'ㄴ', 'ㅣ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㄱ', 'ㅏ', 'ㄴ', 'ㅣ', 'ㅇ', 'ㅎ', 'ㅏ', 'ㄴ', 'ㅣ', 'ㄷ', 'ㄱ', 'ㅣ', 'ㅅ', 'ㅣ', 'ㄱ', 'ㅏ', 'ㄱ', 'ㅏ', 'ㅈ', 'ㅣ', 'ㄱ', 'ㅣ', 'ㅈ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅣ', 'ㅇ', 'ㅌ', 'ㅁ', 'ㅇ', 'ㅣ', 'ㄴ', 'ㅣ', 'ㅈ', 'ㅊ', 'ㄱ', 'ㅊ', '!', 'ㅣ', 'ㄷ', 'ㅣ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㅇ', 'ㅣ', 'ㅣ', 'ㅇ', 'ㅅ', 'ㅌ', 'ㄴ', 'ㅣ', 'ㄴ', 'ㅣ', 'ㅣ', 'ㄷ', 'ㅣ', 'ㄷ', 'ㅣ', 'ㅇ', 'ㅌ', 'ㄱ', 'ㅣ', 'ㅈ', 'ㅊ', 'ㄱ', 'ㅊ', '!', '!', '!']
```

- jamo 라이브러리로 한글 문자열을 자소 단위로 나누어서 반환함.
- 접사와 문장 부호의 의미 학습 ㄱㄴ
- but 개별 토큰은 의미 X, 각 토큰의 의미를 조합할 필요가 있음

형태소 토큰화

1. 형태소 토큰화
 - 형태소 토큰화 : 텍스트를 형태소 단위로 나누는 토큰화 방법 > 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업
 - 문장 내의 형태소 역할 파악 >> 문장 이해하고 처리 ㄱㄴ
2. 형태소 어휘 사전
 - 형태소 어휘 사전 : 각 단어의 형태소 정보를 포함하는 사전 >> 기존의 학습된 어휘 사전에서 새로운 문장이 들어올 때 이를 바탕으로 어휘를 쉽게 학습 ㄱㄴ
 - POS : 품사
 - POS tagging : 각 형태소에 해당하는 품사를 태깅하는 작업
3. KoNLPy
 - KoNLPy : 한국어 자연어 처리를 위해 개발된 라이브러리 > 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공

(1) Okt : SNS 텍스트 데이터 기반으로 개발

```
#5-4. Okt 토큰화
from konlpy.tag import Okt

okt=Okt()
setence="무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns=okt.nouns(setence)
phrases=okt.phrases(setence)
morphs=okt.morphs(setence)
pos=okt.pos(setence)

print("명사 : ",nouns)
print("구 : ",phrases)
print("형태소 : ",morphs)
print("품사 태깅 : ",pos)

명사 :  ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
구 :  ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 :  ['무엇', '이든', '상상', '할', '수', '있는', '사람', '은', '무엇', '이든', '만들어', '낼', '수', '있다', '.']
품사 태깅 :  [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'), ('있는', 'Adjective'), ('사람', 'Noun'), ('은', 'Josa'), ('무엇', 'Noun'), ('이든', 'Josa'), ('만들어', 'Verb'), ('낼', 'Noun'), ('수', 'Noun'), ('있다', 'Adjective'), ('.', 'Punctuation')]
```

- 명사 추출, 구문 추출, 형태소 추출, 품사 태깅(튜플 반환)

표 5.1 Okt 품사 태깅

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

(2) Kkma : 국립 국어원에서 배포한 세종 말뭉치를 기반으로 학습 → 더 세분화된 품사 태깅 제공

```
# 5-5. 꼬꼬마 토큰화
from konlpy.tag import Kkma

kkma=Kkma()
setence="무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns=kkma.nouns(setence)
setences=kkma.sentences(setence)
morphs=kkma.morphs(setence)
pos=kkma.pos(setence)

print("명사 : ",nouns)
print("문장 : ",setences)
print("형태소 : ",morphs)
print("품사 태깅 : ",pos)

명사 :  ['무엇', '상상', '수', '사람', '무엇']
문장 :  ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
형태소 :  ['무엇', '이', '든', '상상', '하', 'ㄹ', '수', '있', '는', '사람', '은', '무엇', '이', '든', '만들', '어', '내', 'ㄹ', '수', '있', '다', '.']
품사 태깅 :  [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄹ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('는', 'ETD'), ('사람', 'NNG'), ('은', 'JX'), ('무엇', 'NP'), ('이', 'VCP'), ('든', 'ECE'), ('만들', 'VV'), ('어', 'ECD'), ('내', 'VXV'), ('ㄹ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('다', 'EFN'), ('.', 'SF')]
```

- 명사, 문장, 형태소, 품사 추출 but 구문 추출 X

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대동 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 물론, 빗금
JX	보조사	SO	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

4. NLTK

- NLTK : 자연어 처리를 위해 개발된 라이브러리 >> 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감정 분석 기증을 제공
- punkt 모델과 averaged_perceptron_tagger모델 활용

(1) 영문 토큰화(punkt 모델)

5-7. 영문 토큰화

```
from nltk import tokenize
```

```
sentence="Those who can imagine anything, can create the impossible."
```

```
word_tokens=tokenize.word_tokenize(sentence)
sent_tokens=tokenize.sent_tokenize(sentence)
```

```
print(word_tokens)
print(sent_tokens)
```

```
['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible', '.']
['Those who can imagine anything, can create the impossible.']
```

- word_tokenize : 단어 토큰라이저, 공백 기준으로 단어 분리 & 구두점을 처리 >> 단어를 추출해 리스트로 반환
- sent_tokenize : 문장 토큰라이저, 구두점을 기준으로 문장 분리해 리스트로 반환

(2) 영문 품사 태깅 → averaged_perceptron_tagger 모델

5-8. 영문 품사 태깅

```
from nltk import tag
from nltk import tokenize
```

```
sentence="Those who can imagine anything, can create the impossible."
```

```
word_tokens=tokenize.word_tokenize(sentence)
pos=tag.pos_tag(word_tokens)
```

```
print(pos)
```

```
[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), (',', ','), ('can', 'MD'), ('create', 'VB'), ('the', 'DT'), ('impossible', 'JJ'), ('.', '.')]
```

- pos_tag : 품사 태깅, 앞서 토큰화한 단어들이 들어가야함

표 5.3 Averaged Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

5. spaCy

- 사이썬 기반 오픈 소스 라이브러리, 머신러닝 기반의 자연어 처리 라이브러리로 빠른 속도와 높은 정확도가 추구미

```
# 5-9. spaCy 품사 태깅
import spacy

nlp=spacy.load("en_core_web_sm")
sentence="Those who can imagine anything, can create the impossible."
doc=nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3} : {token.text}]")

[PRON - DT : Those]
[PRON - WP : who]
[AUX - MD : can]
[VERB - VB : imagine]
[PRON - NN : anything]
[PUNCT - , : ,]
[AUX - MD : can]
[VERB - VB : create]
[DET - DT : the]
[ADJ - JJ : impossible]
[PUNCT - . : .]
```